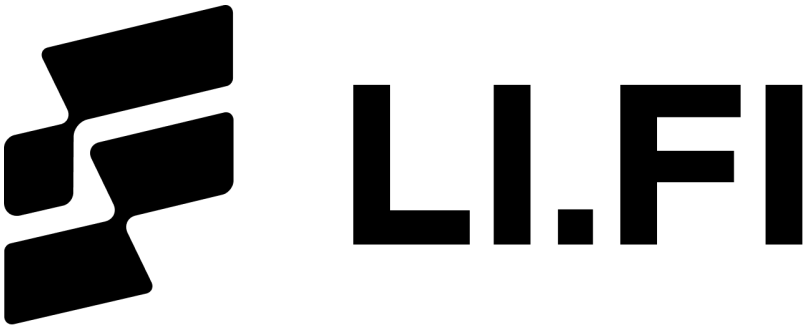

LI.FI Security Review

MayanFacet.sol(v2.0.0)

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

July 22, 2026

Contents

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Facets/MayanFacet.sol(v2.0.0)
- src/Interfaces/IMayan.sol(v1.1.0)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

High leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.

Medium global losses <10% or losses to only a subset of users, but still unacceptable.

Low losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

High almost certain to happen, easy to perform, or not easy but highly incentivized

Medium only conditionally possible or incentivized, but still relatively likely

Low requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Critical Must fix as soon as possible (if already deployed)

High Must fix (before deployment if not already deployed)

Medium Should fix

Low Could fix

5 Executive Summary

Over the course of 7 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

This is an differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of 11 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	37c13a2af
Audit Timeline	July 21,2026 - July 22, 2026
Methods	Manual Review
Documentation	High
Test Coverage	Medium - High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	3
Low Risk	5
Gas Optimizations	0
Informational	3
Total Issues	**11**

6 Findings

6.1 Medium Risk

6.1.1 Native swap calldata is reused with a different input amount

Context: MayanFacet.sol#L143-L144, MayanFacet.sol#L234-L235, MayanFacet.sol#L236-L237

Description: A native double-swap route contains two independent swaps. LI.FI first swaps the user's source asset into native currency, after which Mayan's Forwarder uses `swapData` to exchange that native currency for a middle token. Mayan produces this opaque router calldata for a specific quoted native input amount.

After the LI.FI swap, the facet replaces `_bridgeData.minAmount` with the realized native output and normalizes it. It then passes that new value to `swapAndForwardEth` as both `amountIn` and `msg.value`, but forwards the original `swapData` unchanged. The amount sent to the router can therefore differ from the amount encoded in its calldata whenever the LI.FI swap realizes positive slippage.

Router behavior is implementation-dependent. A router that requires `msg.value` to equal its encoded input amount will revert, making an otherwise valid double-swap route unavailable. A router that consumes only the encoded amount can return or leave the difference with Mayan's Forwarder. Because the nested swap executes with the Forwarder as `msg.sender`, that native surplus does not return to the LI.FI Diamond and is invisible to `refundExcessNative`.

For example, assume Mayan quotes a nested swap of 1 ETH and produces `swapData` that encodes a 1 ETH input with `minMiddleAmount` equal to 1,900 USDC. The preceding LI.FI swap then realizes 1.1 ETH. The facet calls `swapAndForwardEth` with `amountIn = 1.1 ETH` and `msg.value = 1.1 ETH`, while the router calldata still describes a 1 ETH trade. A strict router reverts because the supplied value does not match the encoded input. A permissive router may consume the encoded 1 ETH, return or retain the remaining 0.1 ETH at Mayan's Forwarder, and still produce at least 1,900 USDC, allowing the bridge to continue while the user's 0.1 ETH remains outside LI.FI's refund path.

This issue is independent of stale output floors. Even if `minMiddleAmount` and the downstream order minimum were scaled correctly, the router would still receive calldata and native value describing different trades.

Recommendation: Carry an explicit `mayanAmountIn` that represents the exact amount used to generate `swapData`. After `_depositAndSwap`, require the realized native output to be at least this amount, refund any excess to `refundRecipient`, and call `swapAndForwardEth` with exactly `mayanAmountIn`.

Perform this accounting before normalization so the amount bound to opaque calldata is never rounded or otherwise changed. The backend should construct `swapData`, `mayanAmountIn`, and `minMiddleAmount` from the same Mayan quote and submit them as one internally consistent set.

LI.FI: Fixed in a856296. `swapAndForwardEth` now receives exactly `mayanAmountIn` (raw, matching the amount `swapData` was quoted for) instead of the normalized realized amount, and the surplus is refunded to `refundRecipient`. The router calldata and the forwarded native value therefore always describe the same trade.

Researcher: Verified fix.

6.1.2 Mayan's protocol refund recipient is not validated

Context: MayanFacet.sol#L59-L60, MayanFacet.sol#L188-L189, MayanFacet.sol#L289-L290

Description: The new `MayanData.refundRecipient` field is used for refunds that occur while funds are still inside the Diamond: excess native value and leftovers from the LI.FI source swap. It does not control refunds issued later by Mayan. Mayan stores that recipient independently inside `protocolData`, for example as the Swift order's `trader` or MayanSwap's `refundAddr`.

`_startBridge` parses and validates the destination receiver, but it never reads the corresponding protocol-level refund identity. As a result, calldata can correctly name the user as the destination receiver and pass every facet check while naming a different account for cancellation or expiry refunds.

This is particularly relevant to relayed integrations. If the quote is generated with the relayer, the Diamond, or the Mayan Forwarder as the swapper address, the normal bridge can succeed, so the configuration error may remain

unnoticed. When a later order expires or is cancelled, however, the principal is returned to the address embedded in the Mayan order rather than `MayanData.refundRecipient`. At that point the Diamond cannot redirect or recover the refund. A compromised quote source could use the same gap to direct a deliberately unfillable order's refund to an attacker.

The funding user must still authorize the transaction, so this is not an arbitrary third-party token drain. Nevertheless, it falls under the same untrusted-calldata model as the existing destination-receiver validation and can expose the full bridged principal when the asynchronous refund path is exercised.

Recommendation: Add selector-aware parsing for the source-side refund identity and compare it with `MayanData.refundRecipient` before depositing assets or executing swaps. The parser must deliberately cover every accepted Mayan selector and use the correct address representation for each protocol.

Reject any selector whose refund semantics cannot be validated safely. If maintaining all selector layouts on-chain is impractical, require a trusted backend signature that commits to `BridgeData`, `refundRecipient`, and `keccak256(protocolData)` so a relayer or third-party integrator cannot substitute the refund destination.

LI.FI: Acknowledged. Comparing the order's source-side refund identity (trader/refundAddr) with `MayanData.refundRecipient` does not close this gap: both are populated from the same untrusted calldata, so the comparison only enforces internal consistency and a compromised quote source could set both to an attacker.

It is also breakin, in relayed flows refundRecipient is the source-side funder (e.g. the relayer) while the order's trader is the destination beneficiary, so they legitimately differ and the check would reject valid orders. The sound fix is a trusted-backend EIP-712 signature over BridgeData + refundRecipient + keccak256(protocolData), which is intentionally out of scope for this facet's current design (the destination receiver remains on-chain-validatable). The funding user authorizes the transaction in all cases.

Researcher: Acknowledged.

6.1.3 Increased Mayan inputs retain stale output floors

Context: MayanFacet.sol#L143-L144, MayanFacet.sol#L165-L166, MayanFacet.sol#L238-L239

Description: The value supplied in `_bridgeData.minAmount` is used as a floor for the LI.FI source swap. Once that swap completes, `_depositAndSwap` returns the full amount received rather than the original floor. The facet then normalizes this realized amount and uses it as Mayan's new input amount.

The Mayan quote was built for the original input and contains output protections derived from that amount. On ERC20 routes, `_replaceInputAmount` increases only the encoded `amountIn`; it does not update the order's `minAmountOut`. On native routes, the same issue affects the order's destination minimum, and `swapAndForwardEth` additionally receives the original `minMiddleAmount` for its native-to-middle-token swap.

For example, an order quoted for 100 USDC in and a minimum of 99 units out can receive 110 USDC after positive source-swap slippage. The facet changes the order input to 110 USDC while its enforceable output remains 99. The additional 10 USDC is therefore not protected by the user's quoted exchange rate. A competitive Mayan auction may return part of that value through bidding, but this is not an on-chain guarantee; in a non-competitive or bypassed auction, the settling driver can capture the full surplus while satisfying the stale minimum. The native nested swap has no auction mechanism protecting its stale `minMiddleAmount`.

This violates the intended proportional relationship between the amount committed to the bridge and its minimum output. It also differs from an ordinary slippage setting: the user agreed to a floor for the original amount, not to the same absolute floor after the input was increased.

Recommendation: The safest approach is to preserve the amount for which the Mayan quote was generated. Record that committed amount before `_depositAndSwap`, refund any excess output to `refundRecipient`, and pass only the committed amount to Mayan. If `_bridgeData.minAmount` is only a floor and not the exact quoted Mayan input, add a separate field that explicitly commits to the quoted input amount.

If the product intentionally bridges positive slippage, update every amount-dependent protection by the same ratio. This includes each selector-specific `minAmountOut` inside `protocolData` and `minMiddleAmount` on the native nested-swap path. The calculation must use the same normalized amount that Mayan ultimately receives.

LI.FI: Fixed in a856296. The realized source-swap output is now bound down to a new MayanData.mayanAmountIn (the amount the Mayan quote was generated for) and the positive slippage is refunded to refundRecipient, so the amount reaching Mayan always matches its quoted minAmountOut/minMiddleAmount. We chose bind-and-refund over proportional-scaling because those floors live inside opaque per-selector protocolData/swapData.

Researcher: Verified fix.

6.2 Low Risk

6.2.1 Dynamic MayanSwap payloads corrupt the input-amount rewrite

Context: MayanFacet.sol#L311-L312, MayanFacet.sol#L492-L493

Description: For `MayanSwap.swap`, `_replaceInputAmount` computes the input location as `protocolData.length - 256`. The input is a static ABI-head argument at byte offset 452, while the total calldata length changes with the dynamic `customPayload`. Any non-empty payload shifts the calculated index, causing the facet to overwrite another field while leaving the actual input amount unchanged.

Supported LI.FI routes currently use an empty destination payload, which limits reachability. However, `doesNotContainDestinationCalls` checks only the caller-supplied `BridgeData` flag and does not prevent mismatched `protocolData` from reaching the faulty rewrite.

Recommendation: Use the selector's fixed ABI-head offset and require `protocolData.length >= offset + 32`, or decode and re-encode each supported selector. Add regression tests with empty and non-empty payloads and assert that only `amountIn` changes.

LI.FI: Fixed in c048a00

Researcher: Verified fix.

6.2.2 HyperCore receiver parsing does not validate payload bounds

Context: MayanFacet.sol#L425-L426, MayanFacet.sol#L445-L446

Description: `_parseHypercoreReceiver` follows the caller-supplied `customPayload` offset and reads 20 receiver bytes without checking the offset, encoded length, or required HyperCore payload size. A short but ABI-valid payload can therefore be padded into the value expected by `_bridgeData.receiver`, pass the facet's validation, and create an order the destination handler cannot execute.

The caller must submit and fund the malformed order, so this is a validation and availability issue rather than a third-party theft path.

Recommendation: Bounds-check the dynamic offset and declared payload length before reading. Once the handler, payload type, and HyperCore destination are recognized, require the complete canonical payload length expected by the destination handler and revert on malformed data instead of falling back to `destAddr`.

LI.FI: Fixed in e40e05c. `_parseHypercoreReceiver` now bounds-checks the caller-supplied customPayload offset and declared length (and requires at least the 20 receiver bytes) before reading. A truncated or out-of-range payload now yields a zero receiver; reverting the caller's receiver check instead of a padded partial read that could be crafted to match `bridgeData.receiver`.

Researcher: Verified fix. The added bounds check can be bypassed by a declared payload length near `2^256` (the unguarded add(`off + 0x24, plen`) wraps past dataLen), and only a 20-byte minimum not the canonical HyperCore payload size is enforced; both residuals remain self-funded availability issues, as the oversized length still reverts in Mayan's own ABI decode.

6.2.3 Direct ERC20 deposits are not bound to the Mayan order amount

Context: MayanFacet.sol#L106-L107, MayanFacet.sol#L221-L222, MayanFacet.sol#L165-L166

Description: `startBridgeTokensViaMayan` forwards `_bridgeData.minAmount` of an ERC20 to Mayan without validating the independently encoded input amount in `protocolData`. If `BridgeData` specifies more than the order

consumes, Mayan's Forwarder pulls the larger amount but the protocol consumes only its encoded amount. The remainder stays in the shared Forwarder because `forwardERC20` has no token-return step.

The swap entrypoint avoids this exact mismatch by rewriting the encoded input amount, but the direct entrypoint has no corresponding check.

Recommendation: Parse the order's input amount and require it to equal `_bridgeData.minAmount` before taking user funds. Unsigned calldata may instead be rewritten deliberately; signed order calldata must be rejected on mismatch because mutation can invalidate its signature.

LI.FI: Fixed in a856296. The direct startBridgeTokensViaMayan ERC20 path now requires the order's encoded input amount to equal the amount Mayan pulls (minAmount); a mismatch reverts, so the remainder can no longer be stranded in the Forwarder. It is enforced after receiver validation and is a no-op on the swap entrypoint, where the encoded amount is already rewritten to the bound amount.

Researcher: Verified fix.

6.2.4 The Mayan destination is not bound to BridgeData

Context: MayanFacet.sol#L189-L190, MayanFacet.sol#L272-L273, MayanFacet.sol#L289-L290

Description: The facet validates the receiver encoded in `protocolData` but, outside the HyperCore special case, never compares the encoded protocol destination with `_bridgeData.destinationChainId`. The same EVM address can therefore pass validation while Mayan settles on a different chain from the one emitted by LI.FI. A contract-only receiver may be unusable at the same address on the actual destination.

HyperCore recognition is also gated by the caller-supplied LI.FI chain ID. A genuine HyperCore order declared under another chain ID falls back to validating Mayan's handler address rather than the receiver in `customPayload`.

Recommendation: Parse the protocol destination with the receiver and compare it against a selector-specific mapping of LI.FI chain IDs to Wormhole chain IDs or Circle domains. Detect genuine HyperCore orders from `protocolData` and require their LI.FI destination to be `LIFI_CHAIN_ID_HYPERCORE`.

LI.FI: Acknowledged. Funds still settle to the receiver encoded in `protocolData`, which is validated on-chain, so a chain mismatch is a routing/metadata issue rather than a loss of funds. The destination chain is part of the backend quote and HyperCore is already special-cased. Maintaining a full selector-specific LI.FI→Wormhole-chain / Circle-domain mapping on-chain is brittle relative to the benefit.

Researcher: Acknowledged.

6.2.5 Native swap residue remains in Mayan's Forwarder

Context: MayanFacet.sol#L234-L235, SwapperV2.sol#L67-L68

Description: `swapAndForwardEth` executes the nested DEX swap from Mayan's Forwarder. Unlike its ERC20 swap path, the Forwarder does not return unspent native input to its caller. Any partial fill, router refund, or native dust therefore remains in the shared Forwarder and can only be recovered by Mayan's guardian.

The Diamond cannot recover this value because `refundExcessNative` observes only the Diamond's balance. This is distinct from stale `swapData`: correctly amount-bound calldata can still leave residue if the router under-consumes its input.

Recommendation: Perform the native-to-middle-token swap inside the Diamond and call `forwardERC20`, or require Mayan's Forwarder to return post-swap native residue to `msg.sender`. Until then, accept only exact-input, non-partial-fill calldata produced by Mayan's quote API.

LI.FI: Acknowledged. This is structural to Mayan's immutable Forwarder, which does not return unspent native and can only be swept by Mayan's guardian. With issue *Native swap calldata is reused with a different input amount* (Native swap calldata is reused with a different input amount) fixed the amount-mismatch residue is eliminated (we forward exactly the amount swapData was built for); any remaining under-consumption is router-side and outside the Diamond's control, and the backend supplies exact-input, non-partial-fill calldata. Moving the native-to-middle swap into the Diamond + forwardERC20 is a rearchitecture disproportionate to the residual risk.

Researcher: Acknowledged.

6.3 Informational

6.3.1 ERC20 normalization dust remains in the Diamond

Context: MayanFacet.sol#L157-L158, MayanFacet.sol#L464-L465, MayanFacet.sol#L222

Description: After an ERC20 source swap, the facet rounds the realized output down to eight-decimal granularity and forwards only the normalized amount. The discarded remainder is produced after `SwapperV2` has completed its leftover sweep, and the final output asset is excluded from that sweep in any case. The remainder therefore stays in the Diamond and can be recovered only through an administrative withdrawal.

Each occurrence is bounded below one unit at eight-decimal precision, so the impact is limited to user-owned dust.

Recommendation: Preserve the raw swap output, calculate the normalized bridge amount, and transfer `rawAmount - normalizedAmount` of the ERC20 to `refundRecipient` before `_startBridge`. First enforce that the final swap asset equals the declared bridge asset.

LI.FI: Acknowledged. The 8-decimal normalization is a Wormhole/Mayan protocol requirement, and the discarded remainder is bounded below one unit at 8-decimal precision (true dust), recoverable via administrative withdrawal. A per-transaction dust-refund transfer would cost more gas than the dust is worth. (On the native forwardEth path the normalization remainder is already returned to refundRecipient by the excess-native sweep.)

Researcher: Acknowledged.

6.3.2 The final swap asset is not required to equal the bridge asset

Context: SwapperV2.sol#L101-L102, MayanFacet.sol#L151-L152, MayanFacet.sol#L220-L221

Description: `_depositAndSwap` returns the balance increase of the last swap's `receivingAssetId`, but the facet interprets that number as `_bridgeData.sendingAssetId` without comparing the assets. With no pre-existing bridge-asset balance, the Mayan call normally reverts. If the Diamond holds that asset, a malformed route can forward the stray balance while leaving the actual swap output behind.

The same stray balances are exposed through broader `SwapperV2` leftover behavior, which limits the incremental impact, but the missing equality check still breaks amount and asset accounting.

Recommendation: Before `_depositAndSwap`, require the last swap's `receivingAssetId` to equal `_bridgeData.sendingAssetId`. Apply the check before any deposit, swap, or external token transfer.

LI.FI: Fixed in 0b4063c. `swapAndStartBridgeTokensViaMayan` now requires the last swap's `receivingAssetId` to equal `_bridgeData.sendingAssetId` before any deposit or swap, so the realized swap output and the declared bridge asset can no longer diverge.

Researcher: Verified fix.

6.3.3 Receiver format is selected by a caller-supplied sentinel

Context: MayanFacet.sol#L184-L185, MayanFacet.sol#L196-L197, MayanFacet.sol#L250-L251

Description: `_startBridge` chooses between full-width `bytes32` validation and 20-byte EVM validation solely from whether `_bridgeData.receiver` equals `NON_EVM_ADDRESS`. It does not verify that this representation matches the destination chain.

A byte-addressed destination can be routed through the EVM branch by supplying the low 160 bits of its real receiver, which suppresses `BridgeToNonEVMChainBytes32` and records an unusable EVM-shaped receiver. Conversely, an EVM destination can use the sentinel branch and be reported as non-EVM. Correct `protocolData` still delivers funds to its encoded receiver, so the primary impact is incorrect validation and event metadata.

Recommendation: Maintain a mapping for enabled Mayan destinations that require full-width receivers. Require `NON_EVM_ADDRESS` plus a non-zero `nonEVMReceiver` for those chains, and reject the sentinel on EVM-addressed destinations.

LI.FI: Acknowledged. The NON_EVM_ADDRESS sentinel is a protocol-wide LI.FI convention; correct protocol-Data still delivers to its encoded receiver, so the impact is limited to which validation branch runs and the emitted event metadata, not fund safety. An on-chain per-chain receiver-width mapping is disproportionate for this.

Researcher: Acknowledged.